

Effectiveness of a Generate–Verify–Repair Pipeline with Batfish for LLM-Generated Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired experiment (cnsms2026-001-gvr-batfish-prereg-v1) that evaluated whether a generate–verify–repair (GVR) pipeline—shared initial LLM candidate (gpt-5-mini) + a deterministic Batfish-style validator + at most one automated LLM repair—reduces deterministic-verifier detected semantic/safety failures on intent→vendor-CLI tasks. Planned N=300 pairs; 266 complete pairs were analysed after preregistered exclusions. Baseline pass-rate = 260/266 (97.744%); guarded (final GVR) pass-rate = 265/266 (99.624%). Paired difference = 0.01879699248; exact McNemar two-sided p = 0.0625. Paired bootstrap ($B=10,000$, seed=1729) 95% percentile CI = [0.0037593984962406013, 0.03759398496240601]. Repair was invoked for 6 tasks; median repair iterations per task = 0. Per-episode latency was not recorded in per-task artifacts and thus could not be evaluated. Under shared_initial_candidate semantics the observed absolute improvement is small and did not meet the preregistered $\alpha=0.05$ decision rule. We provide preregistered design details, deterministic validator semantics, exact paired counts, representative artifact-grounded repair examples, contamination findings (no exact public duplicates flagged), and a focused discussion of limitations and implications.

I. INTRODUCTION

Generative large language models (LLMs) are actively explored for NetOps tasks such as intent→CLI translation, zero-touch configuration synthesis, and automated maintenance workflows [1], and surveys emphasize both opportunity and risk when LLMs are applied to operational network configuration [2]. Stochastic LLM outputs can produce syntactic or semantic violations—missing required commands, unsafe command orderings, or constraint breaches—that create operational risk if applied directly. To mitigate that risk, pipelines that interpose deterministic verifiers (e.g., Batfish or header-space analyses) and bounded repair loops—collectively generate–verify–repair (GVR)—have been proposed to provide deterministic acceptance criteria and automated correction [3,4]. Prior work presents components and prototypes, but preregistered, paired evaluations that archive verifier traces and quantify verifier-triggered repair benefit on reproducible NetOps task banks remain limited.

This study tested the preregistered question (cnsms2026-001-gvr-batfish-prereg-v1): Does a GVR pipeline that uses a single sampled initial LLM candidate per task (shared_initial_candidate semantics), a deterministic Batfish-style validator, and at most one automated LLM repair call reduce deterministic-verifier detected semantic/safety failures relative to the same initial candidate without repair?

Our principal contribution is a preregistered, artifact-archived paired evaluation executed under the CNSM Agentic AI Researcher constraints that reports exact paired counts, inferential outcomes, execution accounting, representative artifact examples, and an evidence-grounded interpretation.

II. RELATED WORK

LLM applications in NetOps span intent-to-configuration translation, automated configuration generation, and AIOps toolchains; several recent benchmark and survey efforts document these applications and their limitations [1,2]. Task-level benchmarks such as NetConfEval examine whether LLMs can synthesize vendor CLI fragments from intent descriptions and show that vendor-specific syntax, sequencing requirements, and implicit ordering constraints remain failure points for unconstrained generation [1]. Broad surveys of LLMs for network operations synthesize common failure modes—hallucination, constraint violations, and brittle prompt dependence—and recommend grounding, verifier integration, and tooling to reduce operational risk [2].

Generate–verify–repair (GVR) architectures have been proposed in adjacent domains (e.g., code generation and regulated document production) to combine a stochastic generator with a deterministic oracle and bounded repair steps; these works articulate architectures and convergence desiderata for achieving reproducible, verifiable outputs from stochastic agents [3]. In NetOps, deterministic network verifiers (Batfish and related header-space analyses) provide concrete, automatable checks for reachability, ordering, and policy constraints; Batfish’s engineering history highlights tradeoffs in rule expressiveness and scalability that directly affect its suitability as a validator in automated loops [4]. Empirical and analytic studies of what LLMs require to synthesize correct router or device configurations emphasize that correct sequencing, vendor dialect handling, and explicit constraint encoding are necessary for high first-pass correctness and motivate verifier-driven repair when those elements are missing in a generated candidate [5].

Taken together, the literature establishes three relevant points for our design: (1) NetOps generation tasks combine syntactic, ordering, and topology-dependent semantic constraints that are poorly captured by surface-level language modeling alone [1,5]; (2) deterministic verifiers can provide a domain-appropriate acceptance criterion but have finite rule

coverage and impose engineering tradeoffs that shape experimental outcomes [4]; and (3) GVR-style pipelines are a pragmatic mitigation pattern that can convert nondeterministic LLM outputs into verifier-acceptable artifacts if the repair stage can correct verifier-reported defects within bounded calls [3]. Our study differs from prior work by executing a preregistered paired test using `shared_initial_candidate` semantics, by archiving raw model and validator traces for auditability, and by reporting exact paired contingency counts together with the preregistered exact McNemar test and a paired bootstrap CI to help interpret small-discordant outcomes.

III. METHODOLOGY

Preregistration and estimand: The study preregistration is `cnsms2026-001-gvr-batfish-prereg-v1` (manifest SHA-256: `46a223492f760950b3d06f475129ce21e52e296c4c2a1e09df3ccd50bec9f891`). Primary estimand: `paired_success_rate_difference_guarded_minus_baseline` under `shared_initial_candidate` semantics. Under those semantics the identical single sampled initial model candidate is used to compute both outcomes: baseline single-shot candidate $\rightarrow$ deterministic validator pass/fail, and guarded outcome $\rightarrow$ same candidate; if it fails, the guarded arm may invoke at most one automated LLM repair call and the final validator pass/fail is recorded. This isolates the effect of the allowed validator-triggered repair step for a fixed initial candidate.

Task bank and sampling: The preregistered task bank deterministically produced 300 intent $\rightarrow$ CLI pairs (150 Cisco-like; 150 Juniper-like) using a seeded synthetic task generator combined with public NetConfEval-style examples; per-task generator ids, seeds, and manifest entries are archived. Inclusion/exclusion, retry, and missingness rules followed the preregistration: provider/API transient failures allowed up to two automatic retries; pairs where both arms failed due to provider/infrastructure errors were excluded from the primary analysis. The execution manifest records `planned_episode_count` = 600 and `completed_episode_count` = 532; missingness and reconciliation artifacts are archived.

Model, orchestration, and sampling disclosure: The declared model was `gpt-5-mini` (execution/model_configuration.json field `declared_model_version` = "`gpt-5-mini`"). Archived configuration fields include `max_output_tokens` = 1024, `maximum_attempts_per_call` = 1, `provider` = `openai_responses`, and `temperature` = `null` (unset). Provider accounting (deterministic_reconciliation.json) records `hosted_model_call_event_count` = 306, `completed_hosted_model_call_event_count` = 272, and `failed_hosted_model_call_event_count` = 34. Because temperature was unset and no centralized deterministic sampling seed was archived for provider calls, nondeterministic sampling cannot be excluded for recorded model calls; we note this explicitly in the Disclosure and Limitations.

Deterministic validator semantics and scoring rules: The binary oracle used across episodes was determinis-

tic_netops_validator_v5. For each candidate the validator performed: (1) CLI syntactic parsing/normalization; (2) per-task required-sequence checks (`expected_sequence` vs `parsed_commands`); (3) constrained-field touch checks (`allowed_fields`, `allowed_touched_fields`); and (4) reachability/constraint validation on the synthetic topology model. Each episode archives `validation_before` and `validation_after` traces containing `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, violation codes, and `normalized_configuration`; the validator's pass/fail output is the primary binary endpoint. The confirmatory scoring rule was '`full_intent_constraint_validation`' as recorded in analysis artifacts.

Repair loop mechanics (bounded, archived): Per the preregistration the repair stage is bounded to at most one automated LLM repair call per task when the initial candidate fails the validator. The repair prompt includes validator failure codes and task constraints and requests a corrected configuration; repair provider traces and `repair_prompt_sha256` are archived when invoked. Stage accounting shows `shared_initial_generation` = 300 and `repair` = 6 (six repair calls executed). Repair artifacts and validator traces for repaired episodes are included in execution/scoring and representative artifacts described below.

Analysis plan and inferential methods: The prespecified primary test is the exact McNemar two-sided test on paired binary outcomes at $\alpha=0.05$. Interval estimation was preregistered as a paired bootstrap percentile CI ($B=10,000$, $\text{seed}=1729$) for the paired difference to quantify magnitude and uncertainty. Missingness handling followed the preregistered '`complete_pair_primary`' rule: both-arm provider failures were excluded from the primary test; archived sensitivity analyses treat excluded episodes conservatively as failures. Secondary estimands included `median_repair_iterations_per_task` and `median_end-to-end_latency`; per-episode latency was not archived and thus could not be evaluated. We included the paired bootstrap CI alongside McNemar because exact McNemar p-values can be discrete and conservative with few discordant pairs, whereas a bootstrap CI summarizes empirical paired-difference variability (see Results for the observed small-discordant behavior).

IV. RESULTS

Execution accounting and missingness: Planned episodes = 600; completed episodes = 532; failed episodes = 68 (execution_manifest.json). After applying preregistered exclusion rules (both-arm provider/infrastructure failures), the complete paired units analysed = 266 (analysis/missingness_summary.csv). Provider and model call accounting (deterministic_reconciliation.json) reports `hosted_model_call_event_count` = 306, `completed_hosted_model_call_event_count` = 272, `failed_hosted_model_call_event_count` = 34, `cache_hit_count` = 0, `cache_miss_count` = 272, and stage_counts showing `shared_initial_generation` = 300 and `repair` = 6.

Primary confirmatory outcomes (complete_pairs = 266): baseline successes = 260; guarded final successes = 265. Paired contingency counts (analysis/paired_contingency_table.csv) are n11 = 260 (both pass), n10 = 5 (guarded pass only), n01 = 0 (baseline pass only), n00 = 1 (both fail). Observed proportions: baseline = 260/266 = 0.9774436090; guarded = 265/266 = 0.9962406015. Paired difference = 0.01879699248. Exact McNemar two-sided $p = 0.0625$ (preregistered $\alpha=0.05$ decision rule $\rightarrow$ not significant). Paired bootstrap percentile CI (B=10,000, seed=1729) = [0.0037593984962406013, 0.03759398496240601] (analysis_log.jsonl archives bootstrap parameters and seed). The differing conclusions between exact McNemar ($p>0.05$) and the bootstrap CI (excludes zero) reflect small discordant counts and discrete p-value behavior: with few discordants (5 vs 0) exact McNemar's two-sided discrete p can be conservative while bootstrap CI summarizes resampled paired-difference variability.

Repair behavior and representative artifact examples: Repair stage calls recorded = 6; median repair iterations per task = 0. Representative episode: task-000008 shared initial candidate (execution/responses/task-000008-shared-initial.txt, sha256 = 89306095...) contained a truncated final token ("interface uplink2\n") and failed validation_before (parsed_command_count mismatch). The guarded final artifact (execution/responses/task-000008-guarded.txt, sha256 = d52b7616f...) contained the corrected sequence including the previously truncated line and the required edge1 commands; its validator trace (execution/scoring/task-000008-guarded.json) shows validation_after with valid = true and no violation codes. All repair invocations record repair_prompt_sha256 and repair_provider_trace_path in the archived scoring artifacts; these traces include the validator failure codes used by the repair prompt and the repaired configuration accepted by the validator.

Contamination and sensitivity analyses: The contamination detector (analysis/contamination_summary.csv) produced no exact public-duplicate flags; therefore no exact-duplicate tasks were excluded for contamination in the primary analysis. A preregistered sensitivity analysis that treats the 34 excluded both-arm episodes as failures (conservative complete $N=300$) yields baseline = 260/300 = 0.866667 and guarded = 265/300 = 0.883333 but retains discordant counts n10=5, n01=0 and exact McNemar $p = 0.0625$, indicating the primary inferential conclusion is robust to the preregistered conservative missingness handling.

Supplementary quantitative artifacts: analysis/condition_summary.csv and analysis/paired_contingency_table.csv are archived and hashed. deterministic_reconciliation.json documents that all deterministic consistency checks passed and provides provider call and stage counts used above. Per-episode validator traces (execution/scoring/*.json) provide normalized_configuration and explicit violation codes for every episode and permit replay or reanalysis of individual failure modes.

V. DISCUSSION

Statistical interpretation and the role of small discordant counts: The paired analysis observed only five discordant pairs favoring the guarded pipeline (n10=5) and zero discordant pairs in the opposite direction (n01=0). Exact McNemar two-sided testing evaluates the symmetry of discordant counts and, for small integer discordant counts, yields discrete p-values; with (b,c)=(5,0) the exact two-sided $p = 0.0625$ as archived. The preregistered paired bootstrap percentile CI (B=10,000, seed=1729) for the paired difference produces a 95% interval that narrowly excludes zero [0.00376, 0.03759]. Both results are reported because they illuminate different aspects of the same data: the exact McNemar p emphasizes a strict null-hypothesis test under paired discrete outcomes, while the bootstrap CI summarizes empirical uncertainty about the magnitude of the paired difference under resampling of the observed complete pairs (bootstrap settings and seed are archived in analysis_log.jsonl). The observed apparent tension ($p>0.05$, CI excluding zero) arises from discreteness and small discordant counts rather than from contradictory raw facts; readers should interpret the pair together: a small positive effect was observed, but it did not meet the preregistered exact-test decision threshold.

Effect size, power, and practical detectability in context: The preregistered power calculation targeted detecting a 0.10 absolute improvement at 80% power with $N=300$ pairs. The observed effect (≈ 0.0188) is far smaller than that target; with complete_pairs = 266 (reduced from planned 300 due to 34 both-arm provider failures) the realized power to detect such a small effect is negligible. In practical terms this experiment demonstrates that when baseline first-pass success is already high ($\approx 97.7\%$), bounded automated repair that can be invoked at most once per failed candidate will rarely be exercised (6 repairs invoked across 300 planned tasks) and thus can only produce small absolute improvements unless the baseline error rate is larger or the repair stage is more permissive (e.g., allow multiple repair iterations or stronger repair strategies).

Artifact-grounded diagnostics of pipeline failure modes: The archived per-episode validator traces permit direct inspection of why candidates failed and how repairs corrected them. Representative diagnostics include: - Truncation/malformed output: task-000008's shared initial candidate shows a truncated terminal line ("interface uplink2\n") that caused parsed_command_count mismatches; the automated repair produced the missing explicit admin down line and then passed the validator. Repair artifacts record both the validator violation codes and the repair prompt used, enabling replay of the exact repair stimulus. - Required-sequence violations: several failure traces indicate missing or misordered commands relative to the task's required_sequence; these are precisely the semantic checks the deterministic validator enforces and which repair prompts referenced when invoked. - Constrained-field touch checks: validator traces include observed_touched_field_count and allowed_touched_fields enforcement; failures of this type reflect field-scope violations

rather than pure syntax and are detectable deterministically by the validator. These concrete, archived diagnostics show that the validator’s rule battery produced meaningful, actionable failure signals that the repair prompt could address in some cases, but also that many initial candidates already satisfied the rule battery and thus were unaffected by the repair stage.

Operational and design implications grounded in execution artifacts: The execution accounting (deterministic_reconciliation.json and provider call logs) shows: `hosted_model_call_event_count` = 306, `completed_hosted_model_call_event_count` = 272, `failed_hosted_model_call_event_count` = 34, `shared_initial_generation` = 300, and `repair` = 6. This instrumentation supports several concrete operational inferences: 1) Low marginal utility of bounded repairs on high-accuracy corpora: with few initial failures, a guarded repair stage invoked at most once yields a small absolute improvement; this suggests teams evaluating GVR in practice should calibrate repair budget (number of allowed repair iterations, repair prompt sophistication) to baseline error rates. 2) Auditability and triage: the archived validator traces and scoring JSONs provide a compact, machine-readable audit trail suitable for operator triage and for post-deployment incident analysis; these artifacts concretely demonstrate how deterministic validation can localize a failure cause (syntax vs sequence vs field scope). 3) Latency and throughput unknowns: per-episode timing was not archived and therefore the preregistered latency estimand could not be evaluated; absent timing data, operational latency tradeoffs between invoking a repair and manual review remain unmeasured in this run.

Threats to validity revisited with artifact evidence: Internal validity: the study respected `shared_initial_candidate` semantics preregistered intent and used identical initial candidates for both arms; as a result, the measured estimand isolates the effect of the validator-triggered repair only. Construct validity: `deterministic_netops_validator_v5` enforces a limited, archived rule battery (syntactic parsing, required_sequence checks, constrained_field checks, reachability on the synthetic topology). Semantic failures outside that battery (for example, subtle policy violations not encoded in the validator) would go undetected and are thus absent from the measured endpoint. External validity: the task corpus combined public NetConfEval-style examples and a seeded synthetic generator limited to two vendor dialect clusters; the manuscript therefore does not claim generality to other vendors, larger topologies, or production heterogeneity. Conclusion validity: provider failures reduced the analyzed N (266 complete pairs) and the small number of discordant pairs limits inference for small effects.

Concrete recommendations for future experiments (artifact-grounded): The archived manifests suggest the following preregistered changes would increase diagnostic value and power in follow-on studies: (a) design arms that manipulate first-pass generation (e.g., constrained prompting, multiple independent draws) rather than relying

solely on `shared_initial_candidate` semantics if the goal is to measure effects of prompting or sampling; (b) allow a bounded sequence of repair iterations (≥ 1) and instrument repair-step efficacy; (c) record per-episode latency (request→generation→validation→repair→final validation) to enable formal latency/operational tradeoff analysis; (d) expand and publish the validator rule battery and add adversarial synthetic tasks specifically crafted to probe known verifier false negatives—per-episode validator traces in this run show that such audits are feasible; and (e) if the target operational environment has high baseline accuracy, power calculations should target smaller effect sizes or alternative estimands (e.g., time-to-triage reduction, human-in-the-loop workload) that better reflect marginal benefits of repair.

Reproducibility and archival diagnostics: All raw responses, provider call traces, validator traces, scoring JSONs, analysis artifacts (paired contingency table, condition summary, missingness summary, contamination summary), `deterministic_reconciliation.json`, `model_configuration.json`, and `analysis_log.jsonl` (bootstrap parameters: $B=10,000$, $\text{seed}=1729$) are archived and hashed in the evidence bundle. These artifacts permit exact reanalysis of the paired contingency, replay of repair prompts with the same validator failure codes, and targeted inspection of individual failure modes. The provider accounting and stage counts enable independent auditors to confirm that the repair stage executed only six times and that 34 provider calls failed and were excluded per preregistered rules.

Synthesis: The experiment provides an artifact-rich, preregistered measurement of the limited marginal benefit of a tightly bounded GVR pipeline under `shared_initial_candidate` semantics when baseline first-pass accuracy is high. The archived traces make clear which failure modes the validator detected, which were amenable to one automated repair call, and where further engineering (expanded validator coverage, richer repair strategies, or different experimental arms) would be required to observe larger absolute benefits.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics: both arms used the same single sampled initial candidate per preregistration; the estimand measures validator-triggered repair benefit for that fixed initial sample and does not evaluate independent first-pass generations or constrained first-pass prompting strategies.
- Missingness and sample reduction: planned $N=300$ pairs; after infrastructure/provider exclusions 266 pairs were complete and 34 both-arm episodes were excluded per preregistered rules, reducing effective sample size and precision.
- Small discordant counts: primary inference used exact McNemar with few discordants ($n_{10}=5$, $n_{01}=0$); conclusions are sensitive to inferential choice and limited power.
- Validator coverage: `deterministic_netops_validator_v5` enforces a finite set of syntactic/semantic/reachability

rules; semantic violations outside its coverage are possible and unmeasured.

- Runtime nondeterminism and sampling: archived execution/model_configuration.json records temperature = null and no deterministic seed; nondeterministic sampling cannot be excluded for recorded model calls.
- H2 latency not evaluated: per-episode end-to-end latency was not present in archived per-task artifacts and therefore the preregistered latency bound (median ≤ 60 s) could not be assessed.
- External validity: task corpus derives from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor-dialect clusters; results do not imply universal benefit across other vendors, larger topologies, or production deployments.

VII. CONCLUSION

We executed a preregistered paired experiment that evaluated a GVR pipeline (gpt-5-mini + deterministic_netops_validator_v5 + ≤ 1 automated repair) on intent $\rightarrow$ CLI tasks under shared_initial_candidate semantics. Across 266 analysed pairs the guarded pipeline improved validator pass rate by 0.01879699248 but did not meet the preregistered exact-McNemar $\alpha=0.05$ decision rule ($p=0.0625$); a paired bootstrap CI ($B=10,000$, $\text{seed}=1729$) narrowly excludes zero. Repair calls were rare (6/300) and median repair iterations per task = 0. Per-episode latency was not archived and could not be assessed. All execution and analysis artifacts (raw responses, validator traces, provider call accounting, and reconciliation manifests) are archived in the evidence bundle to support reanalysis and follow-on work.

References [1] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," in Proc. ACM, 2024, doi: 10.1145/3656296. [2] S. Y. Long, J. Tan, B. Mao, F. Tang, Y. Li, M. Zhao, and N. Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," IEEE Commun. Surveys & Tutorials, 2025, doi: 10.1109/comst.2025.3526606. [3] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN, 2026, doi: 10.2139/ssrn.6754899. [4] M. Brown, A. Fogel, D. Halperin, V. Heorhiadi, R. Mahajan, and T. Millstein, "Lessons from the evolution of the Batfish configuration analysis tool," Proc., 2023, doi: 10.1145/3603269.3604866. [5] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," Proc., 2023, doi: 10.1145/3626111.3628194.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model: gpt-5-mini (execution/model_configuration.json archived; declared_model_version = "gpt-5-mini"). Orchestration

adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Preregistration: cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09df3ccd50bec9f891). Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human role and autonomy boundary: before the autonomy lock a human Research Director selected the topic family and approved pre-lock infrastructure development; after the autonomy lock the registered pipeline autonomously performed literature selection, preregistration, experimental execution, analysis, manuscript drafting, AI peer review simulation, and revision. Nondeterminism disclosure: archived execution/model_configuration.json records temperature = null (unset) and no deterministic sampling seed is archived; therefore nondeterministic sampling of model outputs cannot be excluded for the recorded provider calls. Execution and analysis artifacts, logs, and hash manifests are archived in the evidence bundle referenced in the preregistration.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," 2024, 10.1145/3656296
- [2] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," 2023, 10.1145/3626111.3628194